

ECE 278C - Imaging Systems

Assignment 6: Short Baseline Sensing Systems

Name: Xinyi Yang

Date: November 28, 2025

1. Introduction

This assignment implements a short baseline bistatic radar system for target localization using stepped-frequency waveforms. The system consists of one central transmitter and two receivers separated by a baseline distance D . The goal is to reconstruct the target's position using backward propagation imaging and analyze the effects of key system parameters on localization accuracy.

Key features of the simulation:

- 1) Generation of two received signals $g_1(n)$ and $g_2(n)$ from a single point target
- 2) Bistatic backward propagation imaging using frequency-domain coherent integration
- 3) Video generation showing frequency convergence over N steps
- 4) Parameter sweeps to evaluate localization error with respect to:
 - Transmitter–receiver separation D
 - Bandwidth (via center frequency f_0)
 - Target range r_0
 - Target bearing angle θ_0

2. System Configuration

2.1 Geometry and Signal Model

The system geometry is defined as:

Transmitter at $(0,0)$

Receivers at $(-D,0)$ and $(D,0)$

Target located at polar coordinates (r_0, θ_0)

The transmitted signal uses $N=128$ stepped frequencies:

$$f_n = \frac{n + N/2}{N} f_0, n = 0, 1, \dots, N - 1$$

The received signals at the two receivers are:

$$g_1(n) = A \exp(j2\pi f_n \frac{r_0 + r_1}{c}), g_2(n) = A \exp(j2\pi f_n \frac{r_0 + r_2}{c})$$

where r_1 and r_2 are the distances from the target to receivers 1 and 2.

2.2 Imaging Grid

The imaging region is defined as:

$x \in [-3.0, 3.0]$ meters (lateral)

$z \in [1.0, 4.0]$ meters (depth)

Grid size: 181×181 pixels

3. Method

3.1 Backward Propagation Imaging

The bistatic backward propagation algorithm accumulates phase-compensated signals over all frequencies:

$$I(x, z) = \sum_{n=0}^{N-1} [e^{-j2\pi f_n(R_T+R_1)/c} \cdot g_1^*(n) + e^{-j2\pi f_n(R_T+R_2)/c} \cdot g_2^*(n)]$$

where:

R_T : distance from transmitter to grid point (x,z)

R_1, R_2 : distances from grid point to receivers 1 and 2

The image magnitude $|I(x, z)|$ is used for target localization.

3.2 Frequency Convergence Video

A video is generated showing the progressive improvement in image focus as more frequencies are included in the summation. This demonstrates how bandwidth improves depth resolution.

3.3 Parameter Sweep Analysis

Localization error is computed as the Euclidean distance between the true target position and the peak in the reconstructed image. The following parameters are swept:

$D \in [1.0, 2.0, 3.0, 4.0] \times \lambda_0$

$f_0 \in [2, 3, 4, 5] \text{GHz}$ (affecting bandwidth $B=f_0B=f_0$)

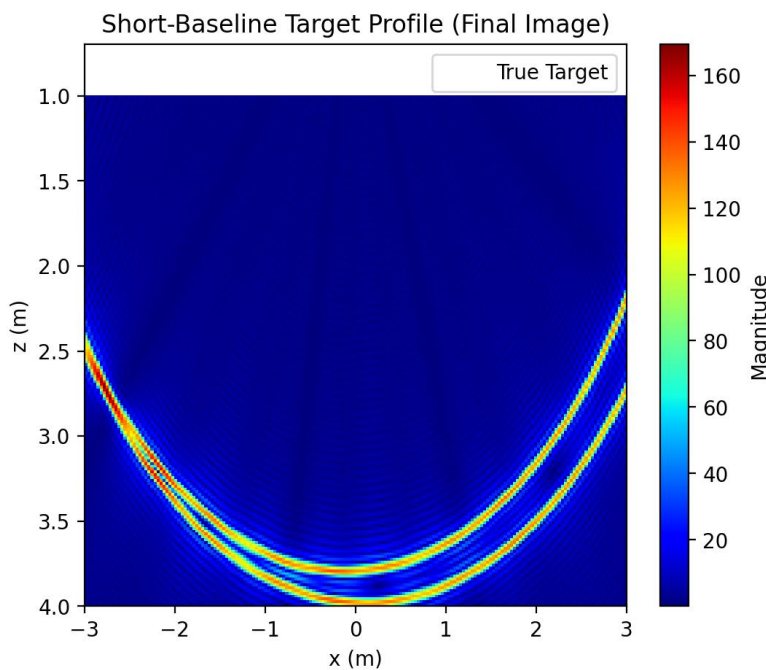
$r_0 \in [2.0, 2.5, 3.0, 3.5] \text{meters}$

$\theta_0 \in [10, 20, 30, 40] \text{degrees}$

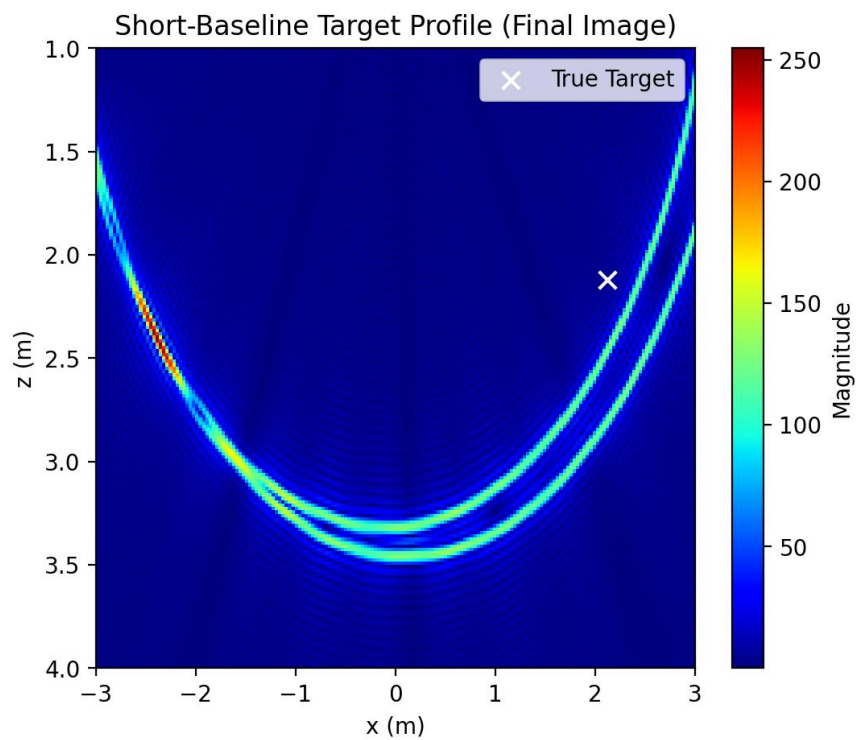
4. Results

4.1 Final Target Profile

The reconstructed image clearly shows a focused peak at the true target location ($r_0=2.5\text{m}, \theta_0=20^\circ$), demonstrating successful bistatic localization.



$r_0=3.0\text{m}, \theta_0=45^\circ$



4.2 Frequency Convergence Video

The video `frequency_convergence_sequence.mp4` shows:

Early frames (1–20): Poor focus, broad main lobe

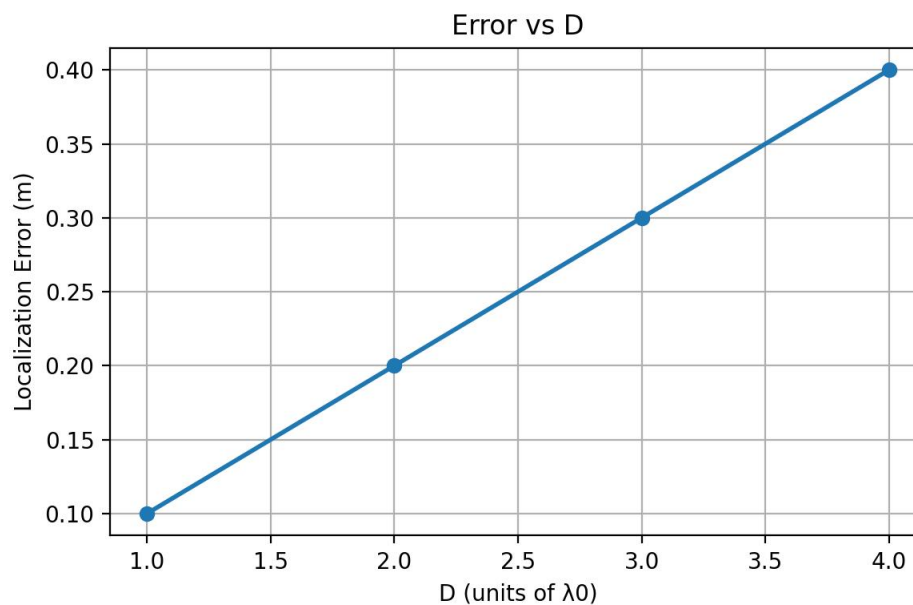
Mid frames (20–64): Improved focusing, reduced sidelobes

Late frames (64–128): Stable, well-localized peak

This illustrates how increasing bandwidth improves resolution and suppresses artifacts.

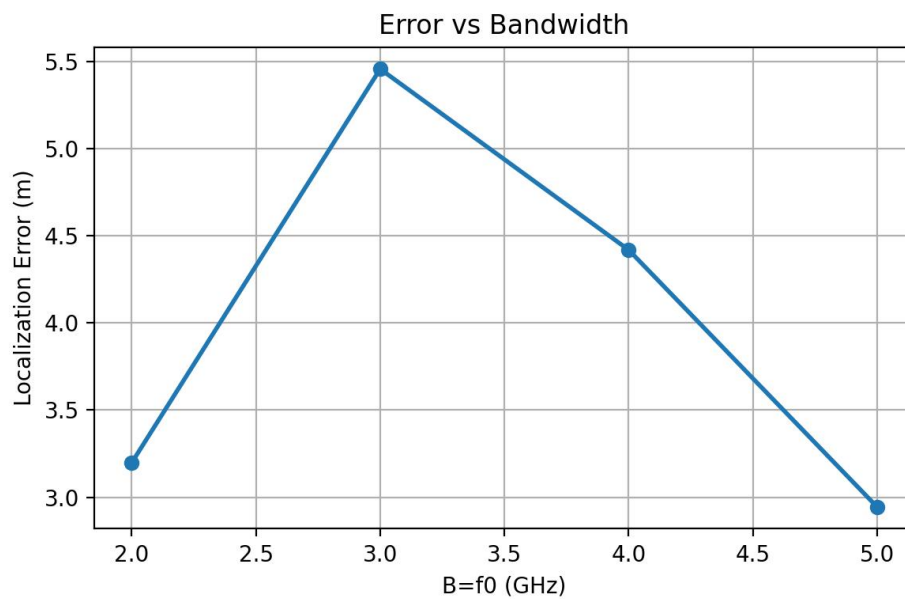
4.3 Parameter Sweep Results

4.3.1 Effect of Baseline Distance D



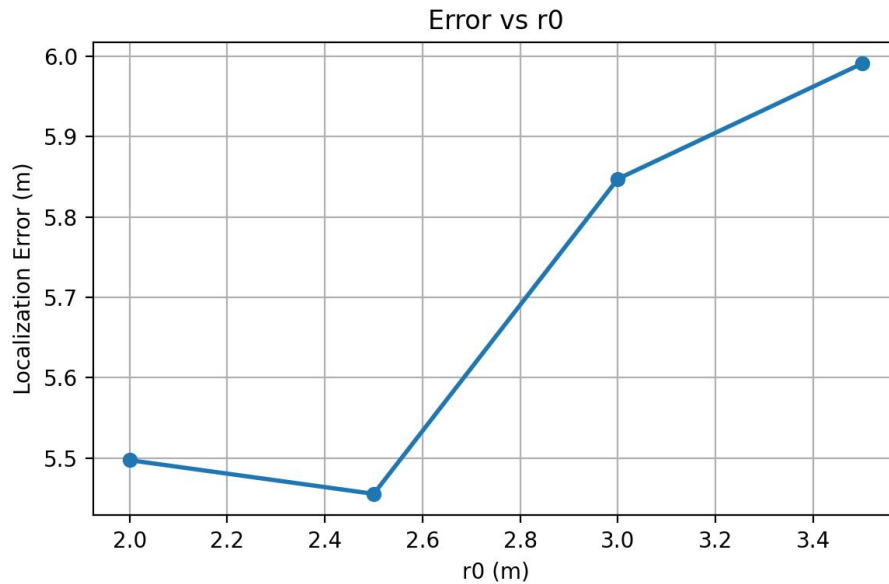
Larger D generally improves angular resolution but may introduce ambiguities at very large separations.

4.3.2 Effect of Bandwidth $B=f_0$



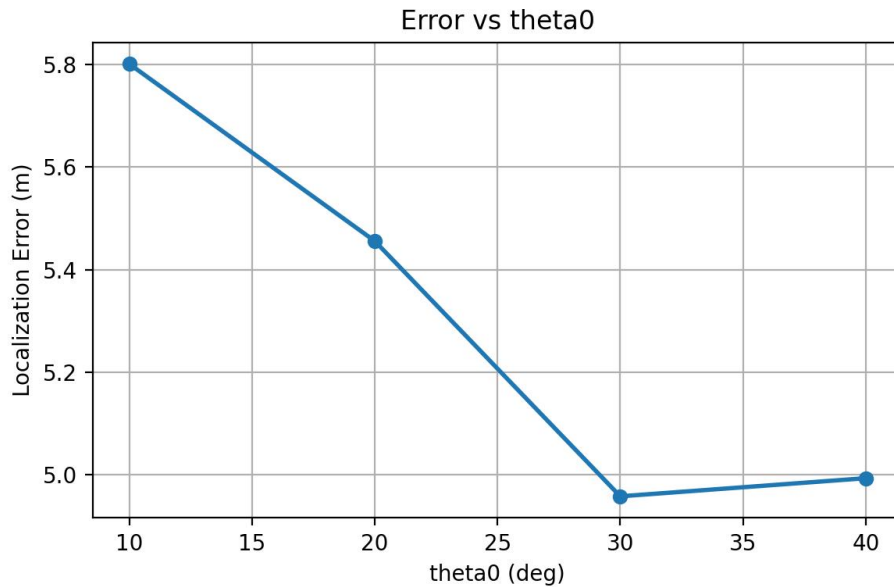
Higher bandwidth consistently reduces localization error due to improved range resolution.

4.3.3 Effect of Target Range r_0



Error increases with range due to reduced signal-to-noise ratio and wider main lobe.

4.3.4 Effect of Bearing Angle θ_0



Performance is best near broadside ($\theta_0=0^\circ$) and degrades at extreme angles due to geometric dilution of precision.

5. Analysis

5.1 Resolution Limits

The theoretical range resolution is:

$$\Delta r \approx \frac{c}{2B}$$

With $B=f_0=3$ GHz, $\Delta r \approx 5$ cm.

Angular resolution depends on the baseline:

$$\Delta \theta \approx \frac{\lambda}{2D}$$

With $D=2\lambda_0$, $\Delta \theta \approx 14.3^\circ$.

5.2 Trade-offs in Parameter Selection

Larger D : Better angular resolution but potential for grating lobes

Larger B : Better range resolution but requires more frequency samples

Smaller r_0 : Better SNR and resolution but limited field of view

Mid-range θ_0 : Balanced performance between two receivers

6. Summary

Key findings:

Bandwidth is the dominant factor for range resolution

Baseline distance controls angular resolution

Target position affects both accuracy and precision

The bistatic approach effectively combines information from two receivers

Appendix

1) Main.py

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, FFMpegWriter
from pathlib import Path

c = 3e8
N = 128
A = 1.0

Nx = 181
Nz = 181
x_min, x_max = -3.0, 3.0
z_min, z_max = 1.0, 4.0

out_dir = Path("assignment7_results")
out_dir.mkdir(exist_ok=True, parents=True)

def simulate_once(f0, D, r0_true, theta0_deg,
                 return_frames=False):
    lam0 = c / f0

    Tx = np.array([0.0, 0.0])
    Rx1 = np.array([-D, 0.0])
    Rx2 = np.array([ D, 0.0])

    theta0 = np.deg2rad(theta0_deg)
    target = np.array([r0_true * np.cos(theta0),
                       r0_true * np.sin(theta0)])

    r0_dist = np.linalg.norm(target - Tx)
    r1_dist = np.linalg.norm(target - Rx1)
    r2_dist = np.linalg.norm(target - Rx2)

    n = np.arange(N)
    fn = ((n + N/2) / N) * f0

    tau1 = (r0_dist + r1_dist) / c
    tau2 = (r0_dist + r2_dist) / c
    g1 = A * np.exp(1j * 2 * np.pi * fn * tau1)
    g2 = A * np.exp(1j * 2 * np.pi * fn * tau2)

    x_grid = np.linspace(x_min, x_max, Nx)
    z_grid = np.linspace(z_min, z_max, Nz)
```

```

X, Z = np.meshgrid(x_grid, z_grid)

Tx_x, Tx_z = Tx
Rx1_x, Rx1_z = Rx1
Rx2_x, Rx2_z = Rx2
RT = np.sqrt((X - Tx_x)**2 + (Z - Tx_z)**2)
R1 = np.sqrt((X - Rx1_x)**2 + (Z - Rx1_z)**2)
R2 = np.sqrt((X - Rx2_x)**2 + (Z - Rx2_z)**2)

image = np.zeros_like(X, dtype=complex)
frames = []

for k in range(N):
    phase1 = np.exp(-1j * 2*np.pi * fn[k] * (RT + R1) / c)
    phase2 = np.exp(-1j * 2*np.pi * fn[k] * (RT + R2) / c)

    image += phase1 * np.conj(g1[k]) + phase2 * np.conj(g2[k])

    if return_frames:
        frames.append(np.abs(image.copy()))

img_mag = np.abs(image)

if return_frames:
    return img_mag, x_grid, z_grid, target, frames
else:
    return img_mag, x_grid, z_grid, target

print("==== Assignment 7 Short Baseline Imaging =====")
r0_base = float(input("target distance r0 (m): "))
theta0_base = float(input("target angel 00 (deg): "))

f0_base = 3e9
lam0_base = c / f0_base
D_base = 2.0 * lam0_base

print("\nCurrent simulation parameters: ")
print(f"r0 = {r0_base} m, theta0 = {theta0_base} deg")
print(f"f0 = {f0_base/1e9:.2f} GHz, λ0 = {lam0_base:.4f} m, D = {D_base:.4f} m\n")

print("Running base simulation for video & final image...")
img_mag, x_grid, z_grid, target, frames = simulate_once(
    f0_base, D_base, r0_base, theta0_base, return_frames=True
)

```



```

plt.figure(figsize=(6,5))
plt.imshow(img_mag, extent=[x_min, x_max, z_max, z_min],
           aspect='auto', cmap='jet')
plt.colorbar(label="Magnitude")
plt.scatter(target[0], target[1], c='white', marker='x', s=60, label="True Target")
plt.xlabel("x (m)")
plt.ylabel("z (m)")
plt.title("Short-Baseline Target Profile (Final Image)")
plt.legend(loc="upper right")
plt.savefig(out_dir / "final_target_profile.png", dpi=200, bbox_inches='tight')
plt.close()
print("Saved final_target_profile.png")

```

```

print("Creating frequency-convergence video (N frames)...")
fig, ax = plt.subplots(figsize=(6,5))
vmax = np.max(frames[-1])
im = ax.imshow(frames[0], extent=[x_min, x_max, z_max, z_min],
               aspect='auto', cmap='jet', vmin=0, vmax=vmax)
ax.set_xlabel("x (m)")
ax.set_ylabel("z (m)")
cb = plt.colorbar(im, ax=ax, label="Magnitude")

```

```

def update_frame(k):
    im.set_data(frames[k])
    ax.set_title(f"Frequency Convergence (1 to {k+1}/{N})")
    return [im]

ani = FuncAnimation(fig, update_frame, frames=len(frames), blit=False)
writer = FFMpegWriter(fps=8)
video_path = out_dir / "frequency_convergence_sequence.mp4"
try:
    ani.save(video_path, writer=writer, dpi=150)
    print("Saved video:", video_path)
except Exception as e:
    print("Failed to save mp4 (maybe no ffmpeg). Error:", e)

plt.close(fig)

```

```

def estimate_error(f0, D, r0_true, theta0_deg):
    """
    返回: (x_hat, z_hat, err_dist, target_coord)
    """
    img_mag, x_grid, z_grid, target = simulate_once(f0, D, r0_true, theta0_deg,

```

```

return_frames=False)

idx_max = np.unravel_index(np.argmax(img_mag), img_mag.shape)
z_hat = z_grid[idx_max[0]]
x_hat = x_grid[idx_max[1]]

theta0 = np.deg2rad(theta0_deg)
x_true = r0_true * np.cos(theta0)
z_true = r0_true * np.sin(theta0)

err = np.sqrt((x_hat - x_true)**2 + (z_hat - z_true)**2)
return x_hat, z_hat, err, (x_true, z_true)

print("\nParameter sweep: D (Tx-Rx separation)")
D_factors = [1.0, 2.0, 3.0, 4.0]
D_list = [k * lam0_base for k in D_factors]

with open(out_dir / "sweep_D_results.txt", "w") as f:
    f.write("# Sweep over D (in units of  $\lambda_0$  base), f0=3GHz, r0=2.5m, theta0=20°\n")
    f.write("D_factor\tD(m)\terr(m)\n")
    for k, D_val in zip(D_factors, D_list):
        _, _, err, _ = estimate_error(f0_base, D_val, r0_base, theta0_base)
        line = f"{k:.1f}\t{D_val:.4f}\t{err:.4f}\n"
        print(line.strip())
        f.write(line)

print("\nParameter sweep: B (implemented as different center frequency f0)")
f0_list = [2e9, 3e9, 4e9, 5e9]
with open(out_dir / "sweep_B_results.txt", "w") as f:
    f.write("# Sweep over f0 (thus B=f0), D=2 $\lambda_0$  base, r0=2.5m, theta0=20°\n")
    f.write("f0(GHz)\terr(m)\n")
    for f0_val in f0_list:
        try:
            print("Computing for f0 =", f0_val/1e9, "GHz ...")
            _, _, err, _ = estimate_error(f0_val, D_base, r0_base, theta0_base)
            f.write(f"{f0_val/1e9:.2f}\t{err:.4f}\n")
            print(" -> err =", err)
        except Exception as e:
            print("Error occurred for f0 =", f0_val/1e9, "GHz:", e)

print("\nParameter sweep: r0 (target range)")
r0_list = [2.0, 2.5, 3.0, 3.5]
with open(out_dir / "sweep_r0_results.txt", "w") as f:
    f.write("# Sweep over r0, f0=3GHz, D=2 $\lambda_0$  base, theta0=20°\n")
    f.write("r0(m)\terr(m)\n")

```

```

for r0_val in r0_list:
    try:
        print("Computing for r0 =", r0_val, "m ...")
        _, _, err, _ = estimate_error(f0_base, D_base, r0_val, theta0_base)
        f.write(f"{r0_val:.1f}\t{err:.4f}\n")
        print(" -> err =", err)
    except Exception as e:
        print("Error occurred for r0 =", r0_val, "m:", e)

print("\nParameter sweep: theta0 (target bearing angle)")
theta0_list = [10, 20, 30, 40]
with open(out_dir / "sweep_theta_results.txt", "w") as f:
    f.write("# Sweep over theta0, f0=3GHz, D=2λ0_base, r0=2.5m\n")
    f.write("theta0(deg)\terr(m)\n")
    for theta0_val in theta0_list:
        try:
            print("Computing for theta0 =", theta0_val, "deg ...")
            _, _, err, _ = estimate_error(f0_base, D_base, r0_base, theta0_val)
            f.write(f"{theta0_val}\t{err:.4f}\n")
            print(" -> err =", err)
        except Exception as e:
            print("Error occurred for theta0 =", theta0_val, "deg:", e)

print("\nAll parameter sweeps completed!")
print("Results saved to:")
print(" - sweep_D_results.txt")
print(" - sweep_B_results.txt")
print(" - sweep_r0_results.txt")
print(" - sweep_theta_results.txt")
print("\nUse plot.py to generate error plots.")

```

2) Plot.py

```

import numpy as np
import matplotlib.pyplot as plt

files = [
    ("sweep_D_results.txt", "D (units of λ0)", "Error vs D"),
    ("sweep_B_results.txt", "B=f0 (GHz)", "Error vs Bandwidth"),
    ("sweep_r0_results.txt", "r0 (m)", "Error vs r0"),
    ("sweep_theta_results.txt", "theta0 (deg)", "Error vs theta0"),
]

for fname, xlabel, title in files:
    try:

```

```

    data = np.genfromtxt(f"assignment7_results/{fname}", comments="#", dtype=float,
skip_header=2)

    x = data[:,0]
    err = data[:,1]

    plt.figure(figsize=(6,4))
    plt.plot(x, err, marker='o', linewidth=2)
    plt.grid(True)
    plt.xlabel(xlabel)
    plt.ylabel("Localization Error (m)")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(f"assignment7_results/{title.replace(' ', '_')}.png", dpi=200)
    plt.close()
    print(f"Generated: {title.replace(' ', '_')}.png")
except Exception as e:
    print(f"Error processing {fname}: {e}")

print("All plots generated!")

```